CS-302 Artificial Intelligence

WEEK 08

AIMAL NAQVI

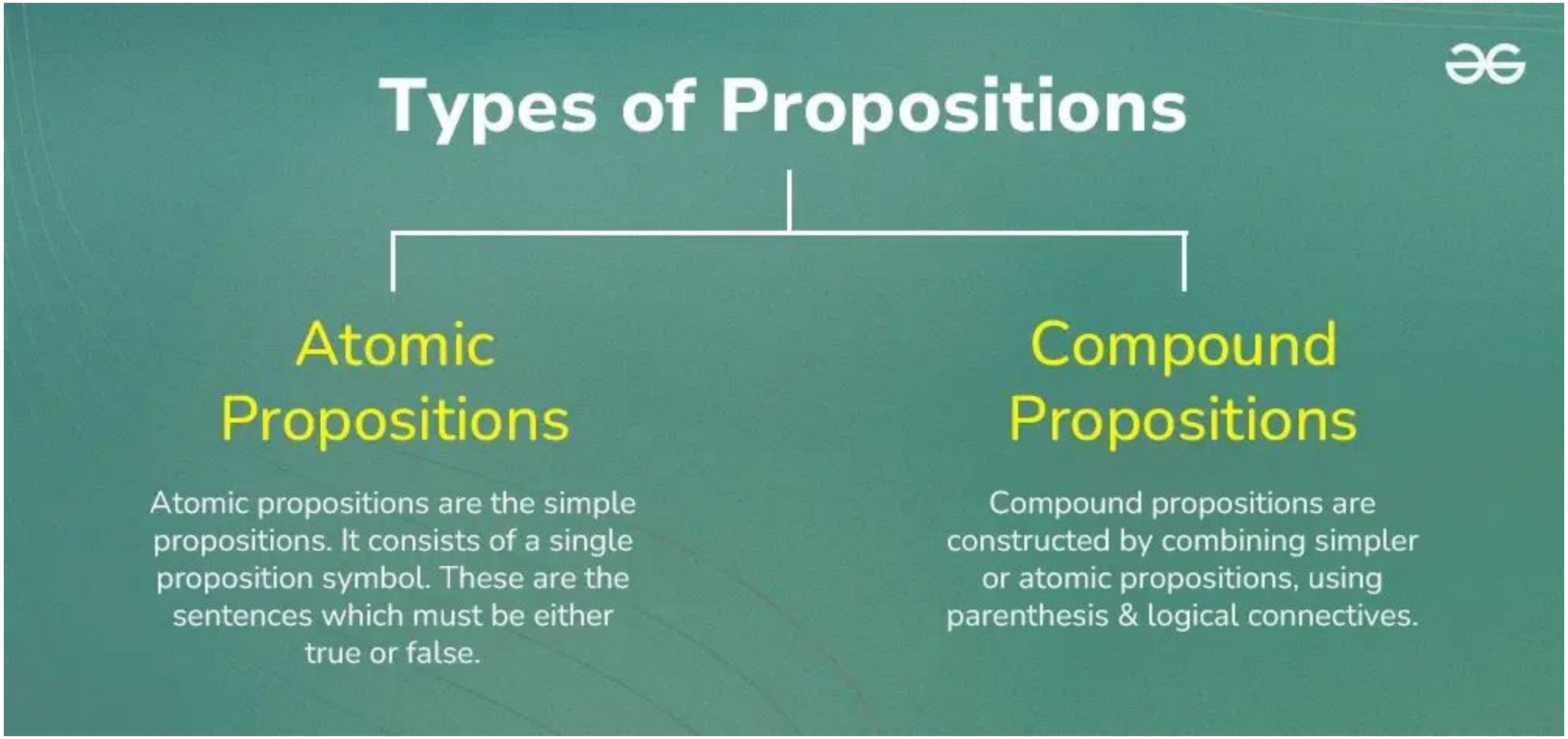
Organization of Lecture

1. Propositional Calculus
2. Set of Equivalence Relation or Laws
3. Rules of Inference
4. First Order Logic (FOL)
5. Unification Algorithm in A.I

Propositional Logic

- Propositional logic is a branch of logic that deals with propositions,
  - Proposition is a declarative statement that can be either true or false but cannot be both at the same time.
  - In AI propositions are those facts or conditions, regarding a particular situation or fact in the world.
  - It is the simplest form of logic used to represent knowledge and reason about it.
  - .

Propositional Logic



Propositional Logic

Examples of Propositions:

- "It is raining."
- "The light is on."
- "Ali is a student."

Each of these can be represented by a single letter:

- Let P = "It is raining"
- Let Q = "The light is on"
- Let R = "Ali is a student"

All these are atomic propositions.

Propositional Logic

- **Logical Connectives:**
  - Connectives are used to combine atomic propositions to form **compound propositions**.
    - **AND (∧)**: True if both propositions are true.
    - **OR (∨)**: True if at least one proposition is true.
    - **NOT (¬)**: Negates the truth value of a proposition.
    - **IF-THEN (→)**: True unless the first proposition is true and the second is false.
    - **IF AND ONLY IF (↔)**: True if both propositions have the same truth value.
    - Parentheses for grouping: (,)

Propositional Logic

- **AND ($\wedge$)**: True if both propositions are true.
  Example: "It is raining AND it is cold." $\rightarrow$ $P \wedge Q$
- **OR ($\vee$)**: True if at least one proposition is true.
  Example: "It is raining OR it is sunny." $\rightarrow$ $P \vee Q$
- **NOT ($\neg$)**: Negates the truth value of a proposition.
  Example: P="It is raining". "It is NOT raining." $\rightarrow$ $\neg P$
- **IF-THEN ($\rightarrow$)**: True unless the first proposition is true and the second is false.
  Example: "If it rains, then the ground is wet." $\rightarrow$ $P \rightarrow Q$
- **IF AND ONLY IF ($\leftrightarrow$)**: True if both propositions have the same truth value.
  Example: "The heater is on IF AND ONLY IF the temperature is low."$\rightarrow$ $P \leftrightarrow Q$.
- These compound propositions help in building more expressive logical statements and rules.

Student Activities

1. Identify the correct logical operator:

   a) If the sun shines, we will go outside → ______

   b) The alarm is set and the door is locked → ______

2. Convert the following sentences into propositional logic:

   a) "If I study, I will pass the test"

   b) "It is raining or it is snowing"

   c) "You can drive if and only if you have a license"

3. Create your own examples using $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$

Equivalence Laws in Propositional Logic

(P, Q, R are propositional variables)

1. Commutative Laws

$$P \land Q \equiv Q \land P$$

$$P \lor Q \equiv Q \lor P$$

These laws say the **order** doesn’t matter for **AND (∧)** and **OR (∨)**.
Just like 2 + 3 = 3 + 2.

Example:

It is cold AND raining” is the same as “It is raining AND cold.

It is cloudy OR it is raining” is the same as “It is raining OR it is cloudy

Equivalence Laws in Propositional Logic

2. Associative Laws

$(P \land Q) \land R \equiv P \land (Q \land R)$

$(P \lor Q) \lor R \equiv P \lor (Q \lor R)$

When combining 3 statements with AND or OR, the **grouping** doesn’t affect the result.
Think: $(A + B) + C = A + (B + C)$

Examples:

(Raining AND Cold) AND Windy” = “Raining AND (Cold AND Windy)

(Cloudy OR Raining) OR Thunder” = “Cloudy OR (Raining OR Thunder)

Equivalence Laws in Propositional Logic

3. Double Negation

$\neg(\neg P) \equiv P$

If you **negate** a statement **twice**, you get back the original.
Saying “It’s not true that it’s not raining” just means “It is raining.”

Equivalence Laws in Propositional Logic

4. De Morgan's Laws

$\neg(P \lor Q) \equiv \neg P \land \neg Q$

Example: It is NOT (raining OR cloudy)" = "It is NOT raining AND it is NOT cloudy.

$\neg(P \land Q) \equiv \neg P \lor \neg Q$

Example: It is NOT (raining AND cold)" = "It is NOT raining OR it is NOT cold.

These laws show how to **distribute a NOT** over AND/OR:

"Not (P or Q)" means "not P and not Q."

"Not (P and Q)" means "not P or not Q."

Equivalence Laws in Propositional Logic

5. Absorption Laws

$P \land (P \lor Q) \equiv P$

Example: It is raining AND (raining OR cloudy)” is also just “It is raining.

$P \lor (P \land Q) \equiv P$

Example: It is raining OR (raining AND cloudy)” is just “It is raining.

These help **simplify** expressions by absorbing the second part:

If P is true, “P AND (P OR Q)” is just P.

If P is true, “P OR (P AND Q)” is also just P

Equivalence Laws in Propositional Logic

6. Law of Contradiction

P	¬P	P ∧ ¬P
1	0	0
0	1	0

P ∧ ¬P ≡ False

This law says a statement **and** its **negation** can **never be true at the same time**.

Always false. You can't say "It is raining AND it is not raining" at the same time.

Equivalence Laws in Propositional Logic

7. Law of Excluded Middle

- **$P \lor \neg P \equiv$ True**

This means **either a statement is true or its negation is true**—there's **no third option**.

Always true. It's either raining or it's not—one must be correct.

P	¬P	P ∨ ¬P
1	0	1
0	1	1

Equivalence Laws in Propositional Logic

8. Law of Idempotency

- **$P \land P \equiv P$**
- **$P \lor P \equiv P$**

Repeating the same statement with AND gives the same result.

P stays the same.

Example: “It is raining AND it is raining” is no different from just saying “It is raining.”

P	$P \land P$
1	1
0	0

Inference Rules

Modus Ponens:

The Modus Ponens rule is one of the most important rules of inference, and it states that if P and $P \rightarrow Q$ is true, then we can infer that Q will be true. It can be represented as:

Notation for Modus ponens: $$\frac{P \rightarrow Q, \quad P}{\therefore Q}$$

Example:

Statement-1: "If I am sleepy then I go to bed" ==> $P \rightarrow Q$
Statement-2: "I am sleepy" ==> P
Conclusion: "I go to bed." ==> Q.
Hence, we can say that, if $P \rightarrow Q$ is true and P is true then Q will be true.

Inference Rules

Modus Tollens:

The Modus Tollens rule state that if P→ Q is true and **¬ Q is true, then ¬ P** will also true. It can be represented as:

Notation for Modus Tollens: $$\frac{P \rightarrow Q, \quad \sim Q}{\sim P}$$

Example:

Statement-1: "If I am sleepy then I go to bed" ==> P→ Q
Statement-2: "I do not go to the bed."==> ~Q
Statement-3: Which infers that "**I am not sleepy**" => ~P

Propositional Logic - Limitations

- Propositional logic can only represent the facts, which are either true or false.
- Not sufficient to represent the complex sentences or natural language statements.
  - "Every student in the room raised their hand.“ Propositional Logic would require a separate symbol for each student, like:
  - P1 = "Ali raised hand", P2 = "Sara raised hand", ...
- Has very limited expressive power PL - **cannot express "some", "all", "none", "every"** types of statements. Consider the following sentence, which we cannot represent using PL logic.
  - "Some humans are intelligent", or
  - “All humans are mortal."

Propositional Logic - Limitations

- Lacks variables, so general rules can't be formed.
  - Example: “If a person is hungry, they eat” $\longrightarrow$ not possible in PL
- Is static — it doesn’t handle dynamic facts.
  - Example: “It is raining now but will stop later” $\longrightarrow$ not possible in PL
- No Internal Structure of Propositions:
  - PL treats entire statements as atomic.
  - You can’t represent or reason about components within a statement.
  - Example: P = "Ali is a student“
  - Can't ask: Who is a student?

First Order Logic

First Order Logic (also called Predicate Logic or First Order Predicate Calculus) is a **formal system** used to describe the world in terms of:

- Objects
- Properties of objects (Predicates)
- Relationships between objects
- Quantified statements like "all", "some"

First Order Logic

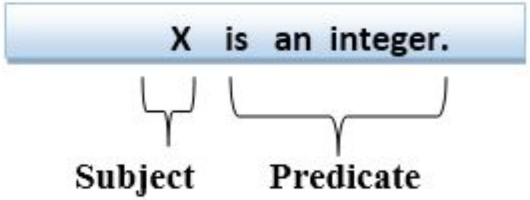
- Whereas propositional logic assumes that a world contains facts, first-order logic, also called Predicate Logic or First Order Predicate Calculus, is a **formal system** (like natural language) that assumes the world contains
- Objects: people, houses, numbers, theories, colors, games, wars, . . .
- Relations: red, round, bogus, prime, multistoried . . ., brother of, bigger than, inside, part of, has color, occurred after, owns, comes between, . . .
- Functions: father of, best friend, third inning of, one more than, end of . . .

First Order Logic

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer.", it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



First Order Logic - Laws

Operator	Symbol	Meaning	Example
Conjunction	$\wedge$	AND	$P(x) \wedge Q(x)$
Disjunction	$\vee$	OR	$P(x) \vee Q(x)$
Implication	$\rightarrow$	IF...THEN	$P(x) \rightarrow Q(x)$
Equivalence	$\leftrightarrow$	IFF	$P(x) \leftrightarrow Q(x)$
Universal	$\forall x$	For all x	$\forall x\ P(x)$
Existential	$\exists x$	There exists an x	$\exists x\ P(x)$

First Order Logic – Laws with examples

Operator	Symbol	Meaning	Natural Language Example
Conjunction	$\wedge$	AND	"x is tall AND smart"
Disjunction	$\vee$	OR	"x is tall OR smart"
Implication	$\rightarrow$	IF...THEN	"If x studies, then x passes"
Equivalence	$\leftrightarrow$	IFF	"x is eligible IFF x is 18 or older"
Universal	$\forall x$	For all x	"All birds can fly"
Existential	$\exists x$	There exists x	"There is someone who won the prize"

First Order Logic - Quantifiers

Quantifiers in First-order logic:

- A quantifier is a language element / symbol which specify the quantity (how many) of instances a statement applies to.
- There are two types of quantifier:
  - **Universal Quantifier, (for all, everyone, everything)**
    - *In universal quantifier we use implication "$\rightarrow$".*
    - $\forall x\ (Human(x) \rightarrow Mortal(x))$
      "All humans are mortal."
  - **Existential quantifier, (for some, at least one).**
  - *In Existential quantifier we always use AND or Conjunction symbol ($\wedge$).*
    - $\exists x\ (Student(x) \wedge Punctual(x))$
      "Some students are punctual.

First Order Logic – Universal Quantifier

Example:

All man drink coffee.

Let a variable x which refers to a **man** v:

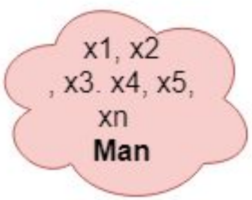
- **x1 drinks coffee**

  ∧
- **x2 drinks**

  ∧
- **x3 drinks milk**

  ∧
- .
- .

  ∧
- **xn drinks milk**



Universe of Discourse

So in shorthand notation, we can write it as :

∀x man(x) → drink (x, coffee).

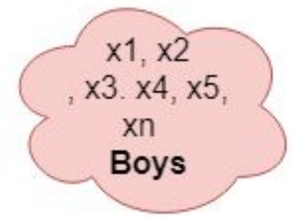
It will be read as: There are all x where x is a man who drink coffee.

First Order Logic – Existential Quantifier

Example:

Some boys are intelligent.

- **x1 is intelligent**
  V
- **x2 is intelligentV**
- **x3 is intelligent**
  V
- **.**
- **.**
  V
- **xn is intelligent**



Universe of Discourse

So in short-hand notation, we can write it as:

∃x: boys(x) ∧ intelligent(x)

It will be read as: There are some x where x is a boy who is intelligent.

First Order Logic - Limitations

More Examples:

- "Every citizen pays taxes." → $\forall x\ (Citizen(x) \rightarrow PaysTaxes(x))$
- "There is a person who knows AI." → $\exists x\ (Person(x) \land Knows(x, AI))$
- All boys like football→ $\forall x\ (Boy(x) \rightarrow Like(x, football))$
- Some boys like football→ $\exists x\ (Boy(x) \land Like(x, football))$

Unification

- Unification is a process of making two different logical atomic expressions identical by finding a substitution. i.e. suitable substitutions for their variables.
- It takes two literals as input and makes them identical using substitution.

Unification

Conditions for Unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
  - Likes(John, x)
  - Hates(John, x)
- Number of Arguments in both expressions must be identical.
  - Likes(John)
  - Likes(John, x)
- Unification will fail if there are two similar variables present in the same expression.
  - Likes(x, x)
  - Likes(John, Mary)

Unification

Type	**Notation / Format**	**Example**
Variables	Start with lowercase or uppercase letters, typically **X, Y, Z**	X, Y, Z, x1, xPerson
Constants	Represent specific fixed objects, usually written as **lowercase names or symbols**	b, john, cat, monday
Functions	Typically lowercase and applied to arguments	f(X), g(b), fatherOf(X)
Predicates	Capitalized, usually name a property or relation	P(x), Loves(John, x)

Unification

- **Variables**: are placeholders that can be substituted.
  Examples: X, Y, Z, x
- **Constants**: are fixed entities or symbols.
  Examples: b, ali, 5, blue, john
- **Functions**: are built from constants/variables and return terms.
  Examples: f(X), g(b), age(ali)

Unification

- p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))

Term	Type	Why?
b	Constant	It's a lowercase fixed symbol
X, Y, Z	Variables	Uppercase letters (placeholders)
f(g(Z))	Function	Function f applied to result of g(Z)
f(Y)	Function	Function f applied to variable Y

Unification

Unify: Knows(John, x) and Knows(y, z)

Substitution: { y/John, z/x }

Result: Both expressions become Knows(John, x)

Unification

. Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

Here, Ψ_1 = p(b, X, f(g(Z))) , and Ψ_2 = p(Z, f(Y), f(Y))

S_0 => { p(b, X, f(g(Z))); p(Z, f(Y), f(Y))}

SUBST θ={b/Z}

S_1 => { p(b, X, f(g(b))); p(b, f(Y), f(Y))}

SUBST θ={f(Y) /X}

S_2 => { p(b, f(Y), f(g(b))); p(b, f(Y), f(Y))}

SUBST θ= {g(b) /Y}

S_2 => { p(b, f(g(b)), f(g(b)); p(b, f(g(b)), f(g(b))} **Unified Successfully.**

And Unifier = **{ b/Z, f(Y) /X , g(b) /Y}**.

{term / variable}
The variable (right side) is replaced.
The term (left side) is what replaces it

Unification Algorithm in A.I

Question:- $\underbrace{Q(a, g(x, a), f(y))}_{A1}$, $\underbrace{Q(a, g(f(b), a), x)}_{A2}$

Substitute: [f(b)/x]

Q(a, g(f(b), a), f(y))
Q(a, g(f(b), a), f(b))

Substitute: [b/y]

Q(a, g(f(b), a), f(b))
Q(a, g(f(b), a), f(b))

Q(a, g(f(b), a), f(b))